Notebook Gemini

Deep Learning for Function Estimation

We'll try to estimate the quality of red wine samples based on their physical and chemical features

We compute and report the mean absolute error, MAE

```
# CNN version
# Regression Example With wine dataset ...
import pandas as pd
from pandas import read_csv
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense , Dropout
from tensorflow.keras.layers import Conv1D
from tensorflow.keras.layers import MaxPooling1D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.models import Sequential
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt

```

```
# mounting the Google drive
from google.colab import drive

drive.mount('/content/drive')

# Load the dataset
fp = "/content/drive/My Drive/Emeritus/Wi20260426/Module3-3/wine_r2_t_noheader.csv"


```

```
Mounted at /content/drive
```

```

# load dataset to a Pandas dataframe
dataframe = read_csv(fp , header=None)
# copy only values to a numpy n dimensional array, ndarray
dataset = dataframe.values

# split into input (X) and output (Y) variables
X = dataset[:,0:11]
Y = dataset[:,11]

print(X.shape)
X = X.reshape(X.shape[0], X.shape[1], 1)
print(X.shape)

```

```
(1599, 11)
(1599, 11, 1)
```

```
print(type(dataset))
print(dataset.shape)
print(dataframe.describe())
print( dataframe.head() )
```

```
<class 'numpy.ndarray'>
(1599, 12)
                 0            1            2            3            4   \
count  1599.000000  1599.000000  1599.000000  1599.000000  1599.000000
mean      8.319637     0.527821     0.270976     2.538806     0.087467
std       1.741096     0.179060     0.194801     1.409928     0.047065
min       4.600000     0.120000     0.000000     0.900000     0.012000
25%       7.100000     0.390000     0.090000     1.900000     0.070000
50%       7.900000     0.520000     0.260000     2.200000     0.079000
75%       9.200000     0.640000     0.420000     2.600000     0.090000
max      15.900000     1.580000     1.000000    15.500000     0.611000

                 5            6            7            8            9   \
count  1599.000000  1599.000000  1599.000000  1599.000000  1599.000000
mean     15.874922    46.467792     0.996747     3.311113     0.658149
std      10.460157    32.895324     0.001887     0.154386     0.169507
min       1.000000     6.000000     0.990070     2.740000     0.330000
25%       7.000000    22.000000     0.995600     3.210000     0.550000
50%      14.000000    38.000000     0.996750     3.310000     0.620000
75%      21.000000    62.000000     0.997835     3.400000     0.730000
max      72.000000   289.000000     1.003690     4.010000     2.000000

                10           11
count  1599.000000  1599.000000
mean     10.422983     5.636023
std       1.065668     0.807569
min       8.400000     3.000000
25%       9.500000     5.000000
50%      10.200000     6.000000
75%      11.100000     6.000000
max      14.900000     8.000000
     0     1     2    3      4     5     6       7     8     9    10  11
0   7.4  0.70  0.00  1.9  0.076  11.0  34.0  0.9978  3.51  0.56  9.4   5
1   7.8  0.88  0.00  2.6  0.098  25.0  67.0  0.9968  3.20  0.68  9.8   5
2   7.8  0.76  0.04  2.3  0.092  15.0  54.0  0.9970  3.26  0.65  9.8   5
3  11.2  0.28  0.56  1.9  0.075  17.0  60.0  0.9980  3.16  0.58  9.8   6
4   7.4  0.70  0.00  1.9  0.076  11.0  34.0  0.9978  3.51  0.56  9.4   5
```

```
# define base model, a deep covolutional one
# structure comprisses 2 1d convolution layers with 16 and 12 filters, filters are 3x1
# then maxpooling for downsampling, a fully connected dense layer with 12 neurons
# Droput layer with 20% at the end
#
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv1D(16, 3, activation="relu", input_shape=(11,1)))
    model.add(Conv1D(12, 3, activation="relu"))
    model.add(MaxPooling1D())
    model.add(Flatten())
    model.add(Dense(12, input_dim=8, kernel_initializer='normal', activation='relu'))
    model.add( Dropout(0.2) )
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model

```

```
# splitting the train and test subsets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
```

```
# training the model for 900 epochs and testing the trained model with the training data
# we report the training error
estimator = baseline_model()
estimator.fit(X_train, Y_train, epochs=420, verbose=2)
prediction = estimator.predict(X_train)

train_error =  np.abs(Y_train - prediction)
mean_error = np.mean(train_error)
```

```
min_error = np.min(train_error)
max_error = np.max(train_error)
std_error = np.std(train_error)
print('\nTraining results:\nmean=',mean_error,'\nmin=',min_error,'\nmax=',max_error,
  '\nstd=',std_error)
```

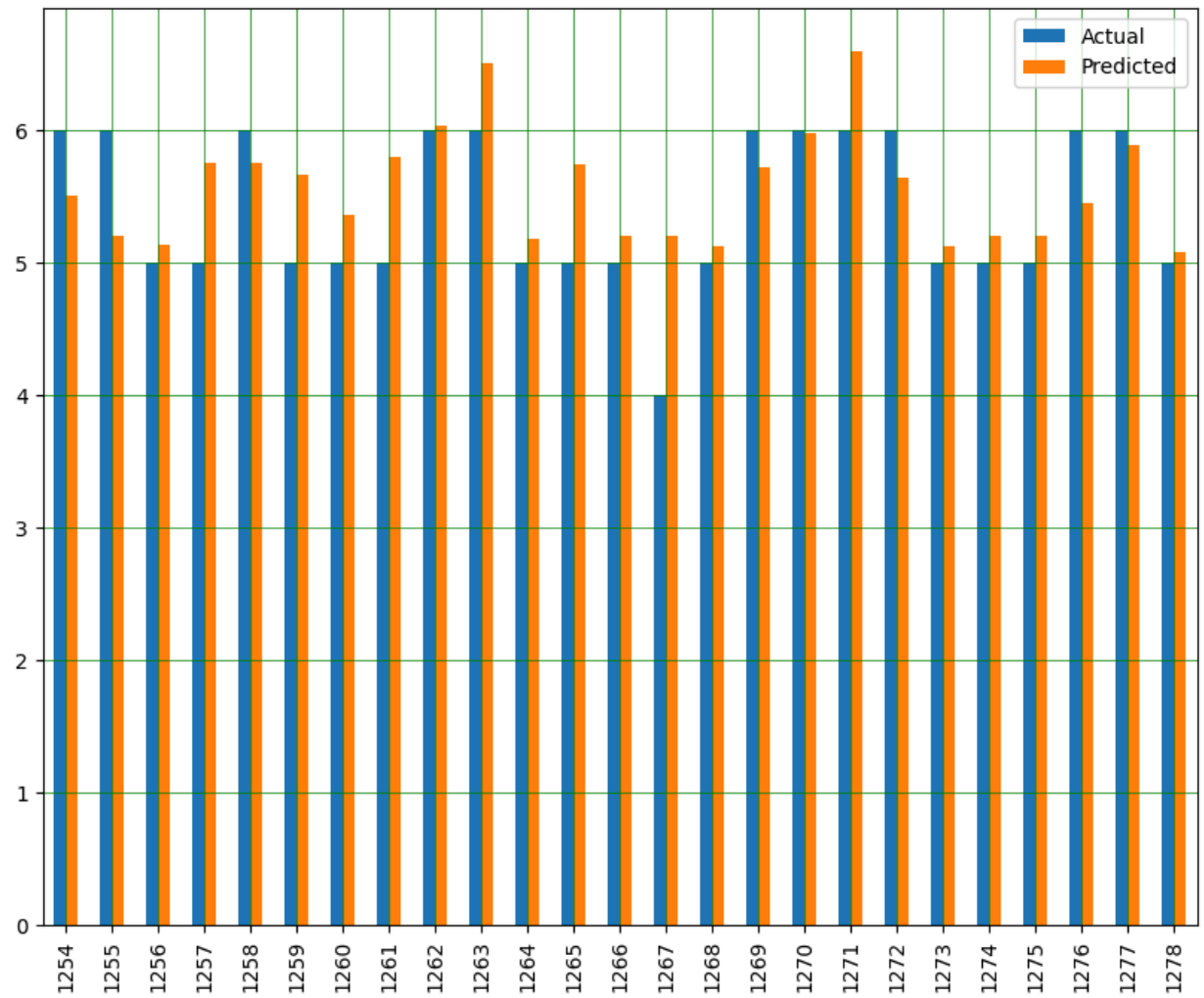
```
Epoch 396/420
40/40 - 0s - 3ms/step - loss: 0.4887
Epoch 397/420
40/40 - 0s - 4ms/step - loss: 0.4873
Epoch 398/420
40/40 - 0s - 3ms/step - loss: 0.5059
Epoch 399/420
40/40 - 0s - 3ms/step - loss: 0.4825
Epoch 400/420
40/40 - 0s - 3ms/step - loss: 0.4893
Epoch 401/420
40/40 - 0s - 3ms/step - loss: 0.4800
Epoch 402/420
40/40 - 0s - 3ms/step - loss: 0.4641
Epoch 403/420
40/40 - 0s - 3ms/step - loss: 0.4894
Epoch 404/420
40/40 - 0s - 4ms/step - loss: 0.4783
Epoch 405/420
40/40 - 0s - 3ms/step - loss: 0.4755
Epoch 406/420
40/40 - 0s - 3ms/step - loss: 0.4728
Epoch 407/420
40/40 - 0s - 3ms/step - loss: 0.4850
Epoch 408/420
40/40 - 0s - 4ms/step - loss: 0.4732
Epoch 409/420
40/40 - 0s - 4ms/step - loss: 0.4777
Epoch 410/420
40/40 - 0s - 8ms/step - loss: 0.4878
Epoch 411/420
40/40 - 0s - 7ms/step - loss: 0.4790
Epoch 412/420
40/40 - 0s - 4ms/step - loss: 0.4725
Epoch 413/420
40/40 - 0s - 8ms/step - loss: 0.4886
Epoch 414/420
40/40 - 0s - 4ms/step - loss: 0.4684
Epoch 415/420
40/40 - 0s - 5ms/step - loss: 0.4935
Epoch 416/420
40/40 - 0s - 4ms/step - loss: 0.4783
Epoch 417/420
40/40 - 0s - 4ms/step - loss: 0.4712
Epoch 418/420
40/40 - 0s - 4ms/step - loss: 0.4670
Epoch 419/420
40/40 - 0s - 4ms/step - loss: 0.4637
Epoch 420/420
40/40 - 0s - 4ms/step - loss: 0.4844
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step

Training results:
mean= 0.7561472388055245
min= 0.0012831687927246094
max= 4.150542259216309
std= 0.5504148572957754
```

```
# showing the real and predicted quality scores for the training data
df = pd.DataFrame({'Actual': Y_train, 'Predicted': prediction.ravel() })
df1 = df.tail(25)
print(df1)
```

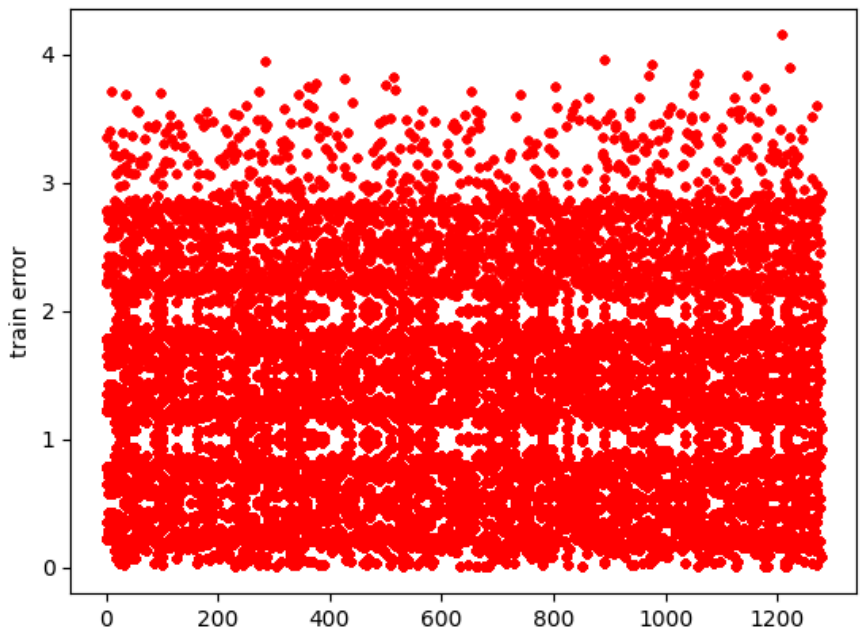
```
      Actual  Predicted
1254     6.0   5.512631
1255     6.0   5.209660
1256     5.0   5.135982
```

```
1257     5.0   5.761653
1258     6.0   5.760108
1259     5.0   5.667025
1260     5.0   5.361681
1261     5.0   5.799149
1262     6.0   6.033411
1263     6.0   6.508229
1264     5.0   5.186313
1265     5.0   5.743219
1266     5.0   5.209660
1267     4.0   5.209660
1268     5.0   5.131938
1269     6.0   5.725645
1270     6.0   5.984919
1271     6.0   6.598526
1272     6.0   5.648352
1273     5.0   5.131783
1274     5.0   5.209660
1275     5.0   5.209660
1276     6.0   5.457989
1277     6.0   5.897164
1278     5.0   5.082387
```

```
df1.plot(kind='bar',figsize=(10,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```




```
plt.plot(train_error,'r.')
plt.ylabel('train error')
plt.show()
```




```
# testing the model with the test data
prediction = estimator.predict(X_test)

test_error =  np.abs(Y_test - prediction)
mean_error = np.mean(test_error)
min_error = np.min(test_error)
max_error = np.max(test_error)
std_error = np.std(test_error)
print('\nTesting results:\nmean=',mean_error,'\nmin=',min_error,'\nmax=',max_error,'\nst
```

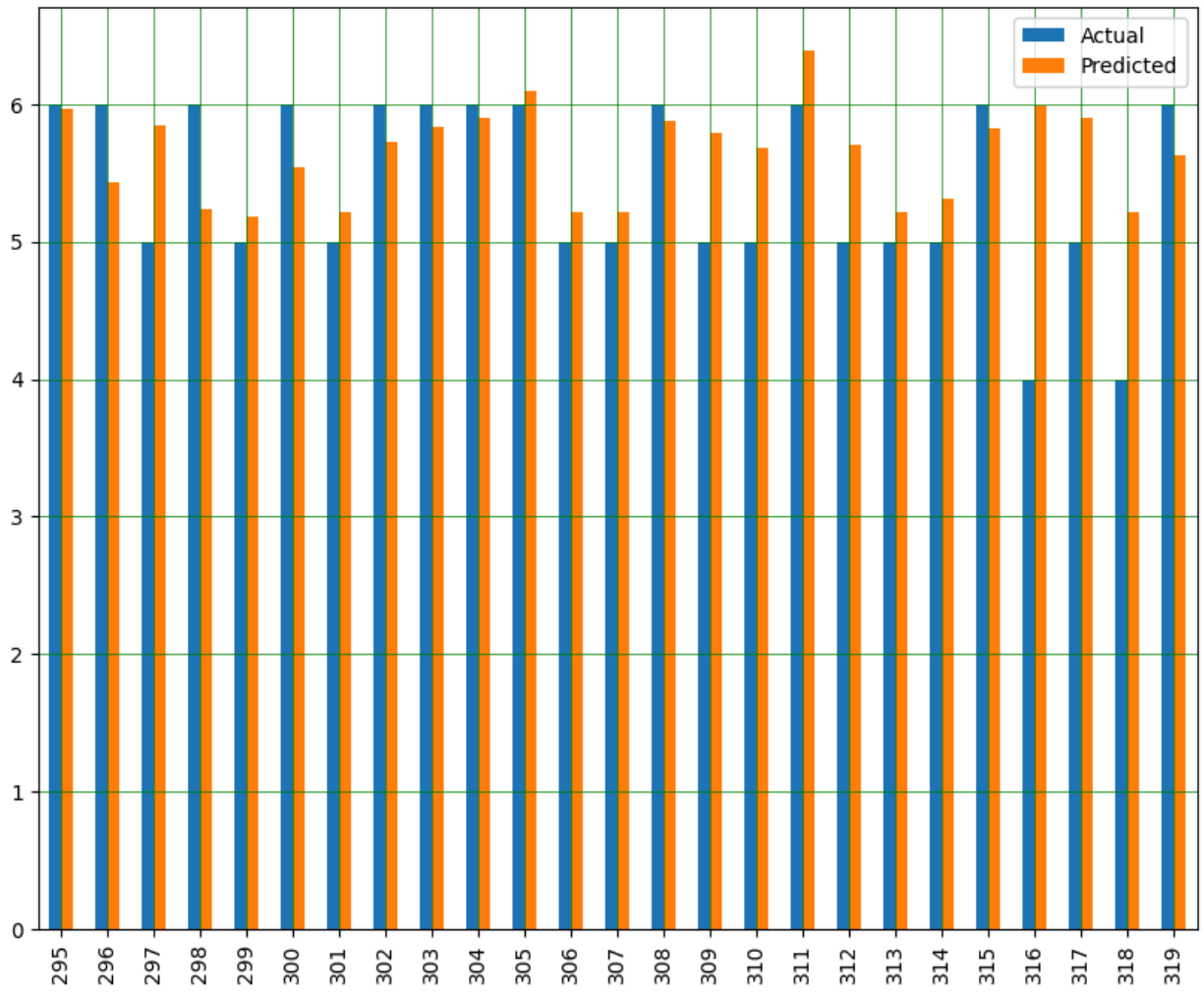
```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

Testing results:
mean= 0.712719029057771
min= 0.013847827911376953
max= 3.947040557861328
std= 0.5255923367005714
```

```
# and showing the results
df = pd.DataFrame({'Actual': Y_test, 'Predicted': prediction.ravel() })
df1 = df.tail(25)
print(df1)
```

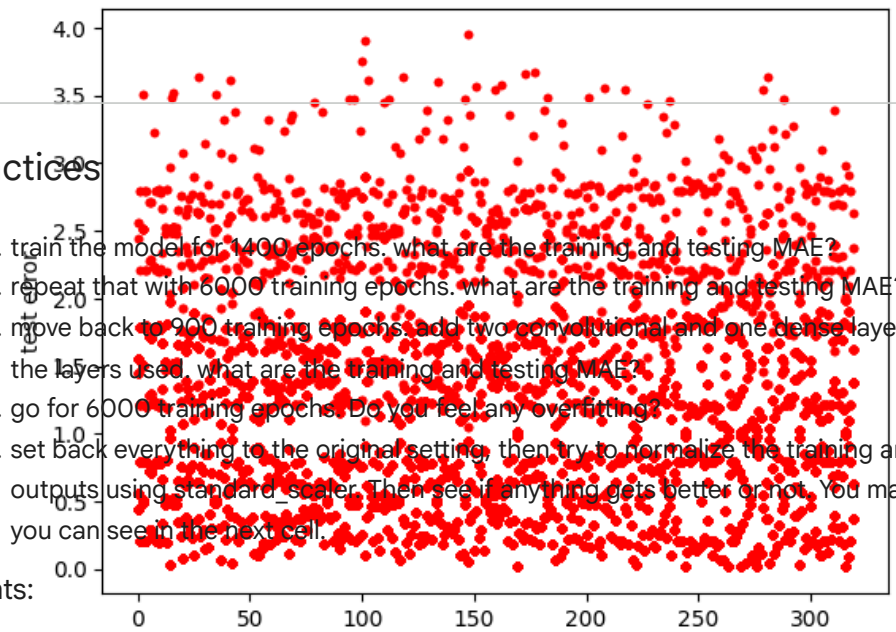
```
     Actual  Predicted
295     6.0   5.970418
296     6.0   5.431983
297     5.0   5.851436
298     6.0   5.235513
299     5.0   5.177028
300     6.0   5.538968
301     5.0   5.209660
302     6.0   5.725477
303     6.0   5.834195
304     6.0   5.901667
305     6.0   6.096517
306     5.0   5.209660
307     5.0   5.209660
308     6.0   5.876126
309     5.0   5.786902
310     5.0   5.685829
311     6.0   6.390170
312     5.0   5.700067
313     5.0   5.209660
314     5.0   5.311828
315     6.0   5.821253
316     4.0   5.986152
```

```
317     5.0   5.906367
318     4.0   5.209660
319     6.0   5.625567
```

```
df1.plot(kind='bar',figsize=(10,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```




```
plt.plot(test_error,'r.')
plt.ylabel('test error')
plt.show()
```



Practices

1. train the model for 1400 epochs. what are the training and testing MAE?
2. repeat that with 6000 training epochs. what are the training and testing MAE?
3. move back to 900 training epochs. add two convolutional and one dense layer with the same features of the layers used. what are the training and testing MAE?
4. go for 6000 training epochs. Do you feel any overfitting?
5. set back everything to the original setting, then try to normalize the training and testing features and outputs using standard_scaler. Then see if anything gets better or not. You may use insttructions like what you can see in the next cell.

points:

- use GPU as your runtime type
- if the notebook overflows, change the verbose to 0, e.g. verbose=0 in the training cell.

```
'''
# normalization
from sklearn.preprocessing import StandardScaler

# Create scalers
scaler_X = StandardScaler()
scaler_y = StandardScaler()

# Standardize X_train and X_test
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

# Standardize Y_train and Y_test (only if y is continuous, e.g., in regression)
y_train_scaled = scaler_y.fit_transform(Y_train.reshape(-1, 1))  # Reshape if needed
y_test_scaled = scaler_y.transform(Y_test.reshape(-1, 1))
'''
```

```
'\n# normalization\nfrom sklearn.preprocessing import StandardScaler\n\n# Create scalers\nsc
aler_X = StandardScaler()\nscaler_y = StandardScaler()\n\n# Standardize X_train and X_test\n
X_train_scaled = scaler_X.fit_transform(X_train)\nX_test_scaled = scaler_X.transform(X_test)
\n\n# Standardize Y_train and Y_test (only if y is continuous, e.g., in regression)\ny_train
_scaled = scaler_y.fit_transform(Y_train.reshape(-1, 1))  # Reshape if needed\ny_test_scaled
= scaler_y.transform(Y_test.reshape(-1, 1))\n'
```